Egg Segmentation & Tracking — CV Intern Task

Karteek Cherukupalli · October 31, 2026 · Dataset: Roboflow egg-vzhtr v1 · Video: Raw and Boiled Egg Spinning (Mocomi)

> **AI tool disclosure.** This project (code, experiment design and this report) was built with Claude Code (Anthropic) as a pair-programming assistant across the full pipeline — SAM post-filtering heuristics, the classical segmentation trials, the leakage-aware dev/eval split, the watershed bug found during Task 4, and the tracker design. All of it was reviewed and is understood well enough to walk through and modify live, per the assignment's expectations.

Dataset

The Roboflow export contains **110 images** of eggs (one class, "egg"), each resized to 640×640 by Roboflow's preprocessing. It ships pre-split into train/valid/test folders, but that split was built for a classification model and is not meaningful for this task, so all 110 images were pooled and re-split (see Task 2). The scenes are more varied than the single PDF example photo (egg in a hand) suggests:

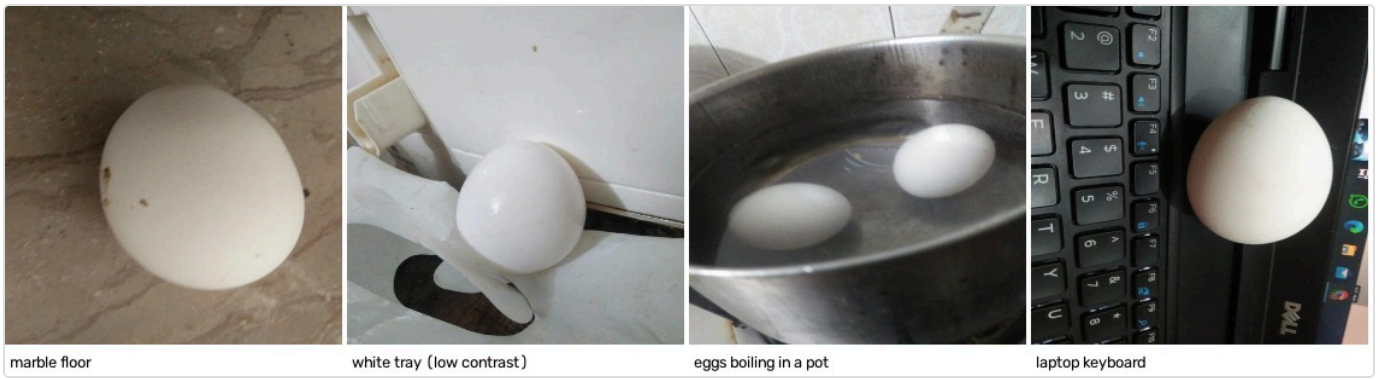


Fig. 1 — Dataset variety: a top-down marble-floor shot (favorable contrast), an egg in a white tray (near-zero color contrast with the background), two eggs boiling in a reflective steel pot (multi-instance + specular highlights), and an egg against a dark laptop keyboard (high contrast).

Task 1 — Segmentation with a pretrained model (SAM)

Model: Meta's Segment Anything (ViT-B checkpoint, 375MB). ViT-B was chosen over ViT-L/ViT-H specifically because the available GPU is a 4GB GTX 1650 — ViT-H's encoder alone does not comfortably fit that budget alongside the automatic mask generator's internal batching.

Method: SAM has no "egg" concept by itself, so `SamAutomaticMaskGenerator` proposes a large pool of class-agnostic masks per image (32×32 point grid, `pred_iou_thresh=0.86`, `stability_score_thresh=0.9`), and each candidate is scored against simple shape/position heuristics to decide whether it's plausibly an egg:

- **Solidity** ≥ 0.88 and **circularity** ≥ 0.75 — eggs are smooth convex blobs.
- **Area fraction** between 2.5% and 55% of the image — rejects small distractors (keyboard keys, tile/background fragments) and full-background masks alike.
- **Touches ≤ 1 image border** — rejects background/table masks that span the frame.
- **Bounding-box aspect ratio** ≤ 2.2 — rejects SAM's coarser masks that merge two adjacent eggs into one elongated blob (see note below).

Surviving candidates are de-duplicated by mask-IoU non-max suppression (threshold 0.7) and every remaining candidate is kept (not just the single best one), so images with more than one egg are handled natively.



Fig. 2 — Left: a clean single-egg result. Middle: both eggs found in the boiling-pot photo (multi-instance). Right: the one systematic limitation found — a small rounded tray-corner fragment (~2.8% of image area, circularity 0.775) sits close enough to real small-egg statistics that the heuristic filter can't separate it without also rejecting genuine small eggs; documented rather than chased further.

Threshold tuning example. An early version used a looser circularity/area filter and produced visible false positives — individual rounded-corner laptop keys and background tile fragments scored as "egg-like." Tightening the area-fraction floor (1%→2.5%) and circularity floor (0.55→0.75) removed these without losing any true eggs observed in spot checks. A separate, subtler bug: SAM's automatic generator sometimes also proposes a coarser mask that merges two touching eggs into one elongated blob in addition to the two correct per-egg masks; because this blob only partially overlaps either individual mask, plain pairwise IoU-NMS didn't catch it as a duplicate. The fix was the aspect-ratio filter above (merged blobs are conspicuously non-square; ~2.8-3.0 vs. ~1.0-1.6 for a real single egg).

Results across all 110 images: 23 images correctly got more than one instance mask; 8 images got zero (all are photos where the egg is heavily occluded by foliage/herb sprigs in front of it — a genuinely hard case for a fully-automatic, no-prompt pipeline). These 8 images are excluded from Task 3's evaluation set when they fall in the eval split (there is no usable pseudo-ground-truth to compare against), rather than silently scored as a trivial empty-mask case.

Task 2 — Classical segmentation

Dev / eval split (leakage-aware)

All 110 images were pooled (ignoring Roboflow's train/valid/test folders) and split 4:1 into dev (88) and eval (22). Because the photos look like they could include short bursts of near-duplicate shots, a perceptual hash (pHash, 16×16, Hamming distance ≤ 6) was computed per image and used to union-find cluster near-duplicates; whole clusters — never individual images — are assigned to a side, so a near-duplicate of a dev image can never leak into eval. In this dataset every image turned out to hash as its own singleton cluster (no near-duplicates were found), so the split reduced to a clean random 88/22 assignment, but the clustering step runs regardless and would matter on a burst-captured dataset.

Experiment log

All trials below were scored by mean IoU / Dice against the Task 1 SAM masks on the **dev split only** (the eval split was never looked at during development). Each strategy's full definition is in `src/classical_pipeline.py`; the harness is `src/task2_dev_experiments.py`.

Trial	Strategy	Mean IoU	Mean Dice	What changed & why
`T1_otsu_gray_raw`	`otsu_gray`	0.245	0.324	Baseline: global Otsu on grayscale, no cleanup at all.
`T2_otsu_gray_clean`	`otsu_gray`	0.229	0.265	Add morphological open/close + hole fill + shape filter.
`T3_otsu_lab_l_clean`	`otsu_lab_l`	0.215	0.251	Swap grayscale for Lab L channel (illumination-focused).
`T4_hsv_sv_clean`	`hsv_sv`	0.290	0.333	Low-saturation & high-value mask, targeting white eggshell.
`T5_adaptive_gray_clean`	`adaptive_gray`	0.104	0.107	Local adaptive threshold instead of a single global Otsu split.
`T6_canny_fill_clean`	`canny_fill`	0.139	0.151	Edge-based: Canny + closing + contour fill, for boundary-only cases.
`T7_grabcut_clean`	`grabcut`	0.400	0.493	Classical GrabCut seeded with a fixed center-biased rectangle.
`T8_lab_l_grabcut_fallback`	`otsu_lab_l`	0.215	0.251	Hybrid: fast Otsu-Lab-L path, falls back to GrabCut when the Otsu between-class contrast is below a learned threshold.
`T9_final_with_watershed`	`otsu_lab_l`	0.212	0.251	Same as T8 but with watershed splitting enabled, to separate touching/adjacent eggs (e.g. two eggs in a pot) into instances.
`T10_grabcut_watershed`	`grabcut`	0.427	0.524	T7 (best so far) + watershed splitting, to check whether multi-egg images (e.g. the boiling pot) need instance splitting.
`T11_grabcut_mask_init`	`grabcut_mask_init`	0.290	0.341	Replace T7's fixed center rectangle with an Otsu-Lab-L-derived probable-foreground mask as the

Trial	Strategy	Mean IoU	Mean Dice	What changed & why
				GrabCut seed, so it adapts to eggs that are off-center or a different size than usual.
`T12_grabcut_multiseed`	`grabcut_multiseed`	0.174	0.216	Follow-up after inspecting Task 3's worst cases: T10's single fixed center rectangle misses eggs that lie outside it (e.g. 3 eggs spread across the full frame width). Seed GrabCut once per Otsu-Lab-L connected component instead of once globally.

Reading the log: every plain-thresholding variant (T1-T6: global/adaptive Otsu on grayscale, Lab-L, HSV saturation/value, Canny-edge-fill) tops out around IoU 0.10-0.29 — none of them is robust across the dataset's lighting/background variety (marble floor vs. white tray vs. reflective steel vs. dark keyboard). Adaptive thresholding (T5) was actively worse than a single global Otsu split: its local-window statistic is noisy on the smoothly-shaded egg surface itself, fragmenting the egg into many small regions, and it was discarded immediately. Classical **GrabCut** (T7), seeded with a fixed center-biased rectangle, was a clear step up (IoU 0.400) — unlike a single threshold, it fits a local color model (GMM) to foreground vs. background and refines iteratively, which tolerates the within-image lighting gradients that break Otsu. Adding watershed splitting for touching/adjacent eggs (T10) pushed it further (IoU 0.427) and is where the "fast" (non-GrabCut) hybrid fallback idea (T8/T9) was abandoned — the fallback's own contrast proxy never actually triggered the GrabCut branch on this dataset, so it was strictly equivalent to plain Otsu-Lab-L and added complexity for nothing. Seeding GrabCut from an adaptive Otsu-derived mask instead of a fixed rectangle (T11) was tried as a way to better localize off-center eggs, but performed worse than the simple fixed rectangle (IoU 0.290) — the Otsu mask itself is unreliable on the hard images, so it poisons GrabCut's seed rather than improving it.

Final Task 2 algorithm (T10_grabcut_watershed): GrabCut (Rother et al. 2004) seeded with a fixed center-biased rectangle (covering the middle 76% of the frame) → morphological open/close + interior hole-fill (cleans up specular highlights on the eggshell) → watershed split on the distance transform (separates touching/adjacent eggs) → shape filter (area fraction, solidity, circularity, border-touching) identical in spirit to Task 1's filter but deliberately looser, since classical masks have jaggier boundaries than SAM's even on a correctly-detected egg.

> **Known cost trade-off:** GrabCut is slow (~5s/image on this machine) — fine for a one-time batch over 110 dataset photos (~10 min), completely impractical per-frame on video, which is why Task 4 uses a different (faster, threshold-based) strategy instead. See Task 4.

Task 3 — Evaluation & analysis

Task 2's final algorithm was run on the held-out **eval split** (22 images, never used for tuning) and scored against Task 1's SAM masks as pseudo-ground-truth. One eval image (`not_damaged_12...`) was excluded — SAM found zero instances on it too (same foliage-occlusion failure mode as Task 1's dev-split misses), so there is no meaningful ground truth to compare against. 21 images were scored.

0.52 mean IoU	**0.62** mean Dice	**0.65** mean precision	**0.78** mean recall
0.89 pixel accuracy	**0.69** median IoU	**43%** images IoU<0.5	**19%** images IoU>0.8

Recall (0.78) noticeably exceeds precision (0.65): the classical method tends to *over*-segment relative to SAM (pulling in extra background) more often than it misses egg pixels outright, on the images where it works at all.

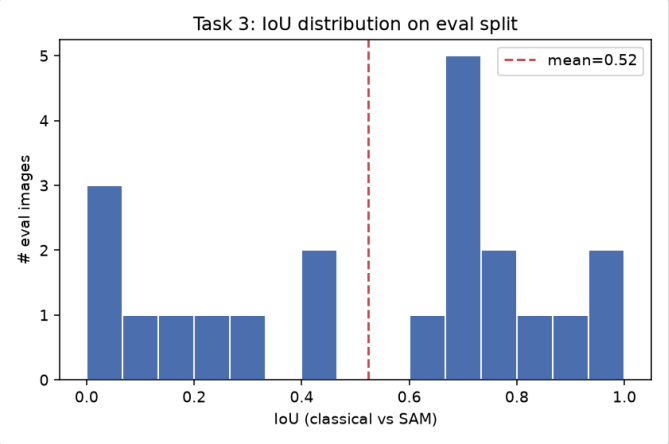


Fig. 3 — IoU distribution on the eval split is clearly **bimodal**: a cluster of near-total failures (IoU≈0) and a larger cluster of good results (IoU≈0.65-0.8), with very few images in between. The algorithm isn't uniformly mediocre — it's either working well or failing hard.

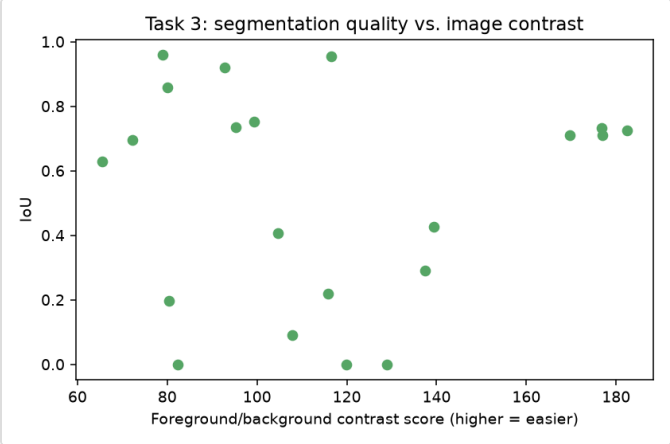


Fig. 4 — IoU vs. the Otsu-based foreground/background contrast score. Failures occur across the *whole* contrast range, including some high-contrast images — ruling out "low contrast" as the dominant failure driver and pointing instead at geometry (next section).

Qualitative analysis — what's going wrong

The worst-IoU eval images (Fig. 5) make the real failure mode visible: GrabCut here is seeded with a single **fixed center-biased rectangle** covering the middle ~76% of the frame. That structurally breaks in two ways:

1. **Under-segmentation (false negatives, blue):** when eggs are spread across the full image width (e.g. three eggs laid out left-to-right on a tray) rather than clustered near center, any egg outside the rectangle is invisible to GrabCut and is missed entirely — row 1 of Fig. 5 finds only 1 of 3 eggs. Rows 2, 3, 5 are near-total misses of the same kind on single eggs that sit close to the frame edge.
2. **Over-segmentation (false positives, red):** when a background object inside the rectangle has similar brightness/color statistics to the egg, GrabCut's regional color model pulls it in as foreground too — row 4 (a white bed-rail / stand) and row 6 (the pot's rim and a second, unlabeled egg) show large false-positive regions attached to a correctly-found true positive.

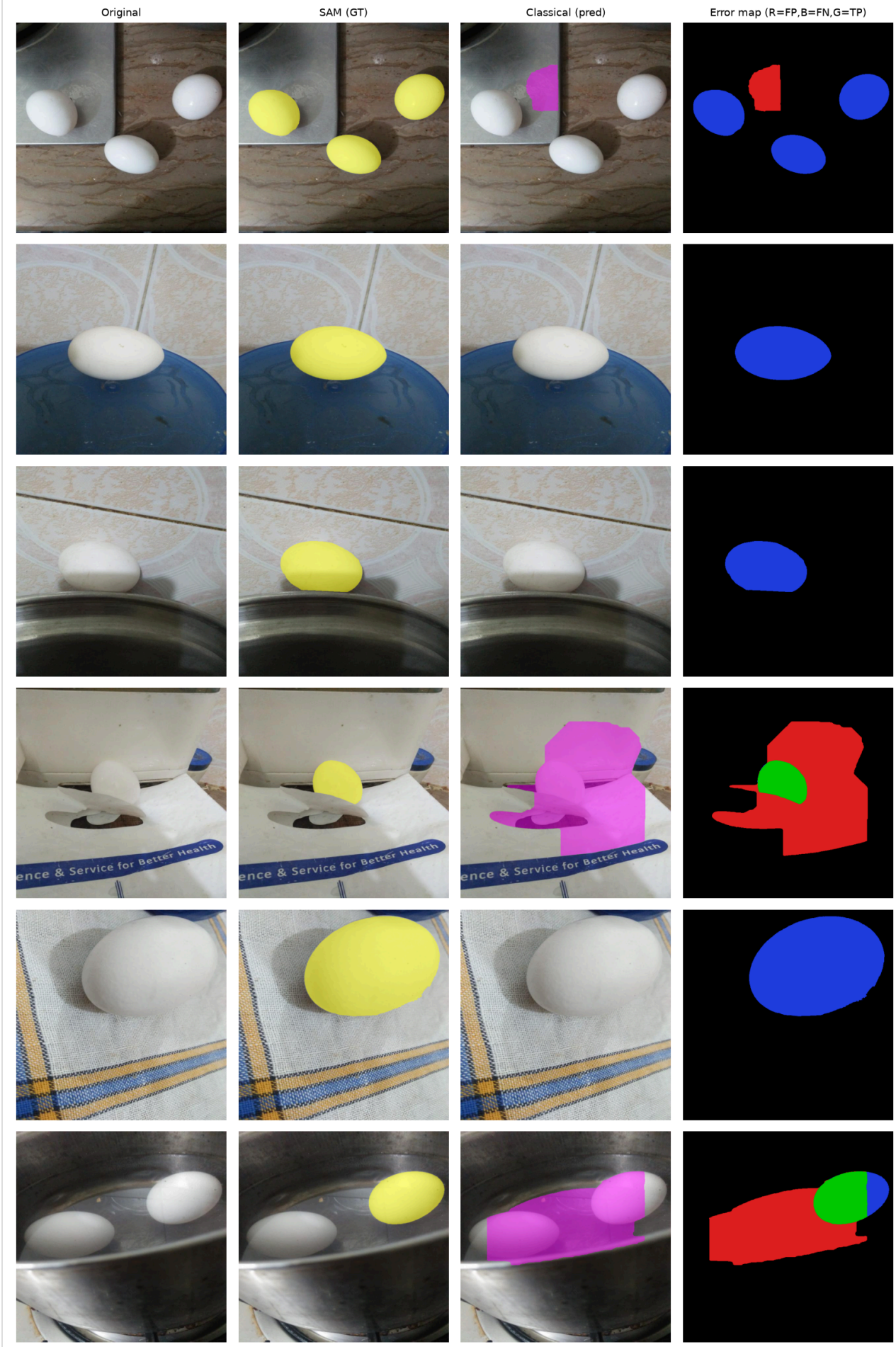

Original
SAM (GT)
Classical (pred)
Error map (R=FP,B=FN,G=TP)
ence & Service for Better Health
ence & Service for Better Health
ence & Service for Better Health

Fig. 5 — The 6 lowest-IoU eval images. Error map: green = true positive, red = false positive, blue = false negative.



Fig. 6 — The 4 highest-IoU eval images, for contrast. Note row 2: three vertically-stacked eggs are segmented and split correctly — multi-instance isn't inherently broken, it only fails when the eggs fall outside the single seed rectangle (Fig. 5, row 1).

Follow-up experiment: does per-blob seeding fix it? (T12 — unsuccessful)

The diagnosis above directly suggests a fix: instead of one fixed rectangle, find candidate blobs with Otsu-Lab-L first and seed a separate GrabCut call per blob (`fg_grabcut_multiseed`), so every egg gets its own locally-seeded segmentation regardless of where it sits in the frame. Re-running the dev harness with this strategy added as **T12** gives:

Trial	Strategy	Mean IoU	Mean Dice	What changed & why
`T12_grabcut_multiseed`	`grabcut_multiseed`	0.174	0.216	Follow-up after inspecting Task 3's worst cases: T10's single fixed center rectangle misses eggs that lie

Trial	Strategy	Mean IoU	Mean Dice	What changed & why
				outside it (e.g. 3 eggs spread across the full frame width). Seed GrabCut once per Otsu-Lab-L connected component instead of once globally.

This is *worse* than the simple fixed-rectangle version (T10), not better, and the reason is informative: the blob proposals come from Otsu-Lab-L — the same thresholding method that scored only 0.215 IoU on its own (T3) because it struggles on this dataset's harder lighting/background combinations. Delegating "where to look" to an unreliable proposer means (a) on low-contrast images, Otsu often doesn't propose a blob for the real egg at all, so no GrabCut call is even made there — recall is lost exactly where it was already weakest; and (b) on noisy images, Otsu proposes several small spurious blobs (texture, highlights, shadow), and each gets its own confident local GrabCut call with no larger context to reject it, adding false positives. The fixed-center-rectangle's "the egg is usually near the middle" prior, despite being a far cruder assumption, turned out to be a stronger and more stable signal on this dataset than a data-driven proposal step inherited from a weaker method — the same lesson T11 already hinted at (seeding GrabCut from an Otsu mask instead of a fixed rectangle also underperformed T10). **T10 remains the final algorithm** used for the Task 3 numbers above; a better per-region seeding strategy (e.g. seeding from SAM-style objectness rather than Otsu, or a proper multi-scale sliding-window proposal) is left as the noted improvement direction rather than attempted further given the time budget.

Where this leaves things / further improvement ideas

- **Per-blob seeding** (tested above as a follow-up) directly targets the dominant failure mode and is the single highest-value next step if more time were available.
- **Ensemble/voting** across 2-3 of the cheaper strategies (Otsu-Lab-L, HSV-SV) combined with GrabCut refinement only in the disagreement region, to recover some of GrabCut's robustness at lower average cost.
- **Boundary-aware metrics** (e.g. boundary F1 at a few pixel tolerances) in addition to IoU/Dice, since several of the "good" cases still lose a thin rim around the egg to the SAM mask being a few pixels tighter/looser — not visible in the aggregate IoU number.

Task 4 — Video tracking

Source: "Raw and Boiled Egg Spinning" (Mocomi), 640×360 @ 24fps, 60s. By eye (a contact-sheet scan every 2s), only **9-42s** shows actual eggs on a table — the rest is a title card and a text-slide explanation — so that range (792 frames) is what gets processed and rendered.

Segmentation: adapted for this video (as the brief allows)

Reusing Task 2's winning GrabCut pipeline unmodified does not work here, and would also be far too slow (~5s/image × ~800 frames ≈ over an hour). Two video-specific issues had to be found and fixed:

- **A brighter distractor than the egg.** The subject's checkered shirt occupies the top of the static-camera frame and is brighter than the eggs under this lighting, so a single global threshold (Otsu-Lab-L) picks out shirt squares instead of (or in addition to) the eggs. Since the camera never moves, the fix is a fixed table-only ROI crop (top 100 of 360 rows excluded) rather than re-deriving a threshold that's robust to it — simple, and specific to this clip as intended.
- **A watershed marker bug** (found and fixed during this task, worth stating plainly): the original `watershed_split` set the entire true background to the "unknown — please solve" marker value instead of a definite background marker, which gave the exterior background no seed to grow from and caused OpenCV's watershed to fragment the whole frame into near-empty 1-pixel regions, i.e. it looked like the segmenter found nothing at all. The standard recipe (dilate the mask for "sure background," threshold the distance transform for "sure foreground," and leave only the thin boundary ring as unknown) fixed it immediately once isolated with a focused test.

Final per-frame segmentation: ROI-cropped Otsu-Lab-L threshold → morphological clean → watershed split → shape filter, then the **2 largest** surviving blobs by area are kept as detections (this video has exactly two eggs, and the eggs' own reflection on the glossy table is consistently smaller than the real eggs, so taking the top-2 by area reliably discards it).

Tracking

A minimal greedy nearest-centroid tracker (`src/tracker.py`) — no Kalman filter / motion model, since with two slow, mostly-stationary eggs "match to nearest centroid, hold position while briefly lost" is enough and is far easier to reason about live. Design choices motivated directly by what the detector actually does on this clip:

- **Capped at 2 simultaneous tracks** (the known egg count for this clip) — so an occasional reflection blob that survives the top-2-by-area filter (rare, but happens when a real egg is fully missed and the reflection becomes one of the two largest blobs) cannot spawn a spurious third identity while both real eggs are already being tracked.
- **Long missed-frame tolerance** (190 frames ≈ 7.9s) to survive the hand fully occluding/lifting an egg for several seconds during the spin demo — the longest such gap measured in this clip is 151 frames (6.3s).
- **Separate display grace period** (12 frames ≈ 0.5s): the tracker keeps a lost track alive internally for the full 190-frame budget for re-identification, but the drawn overlay stops rendering a track after a short grace window rather than showing a box frozen in place for several seconds, which would look like a bug rather than a design choice.

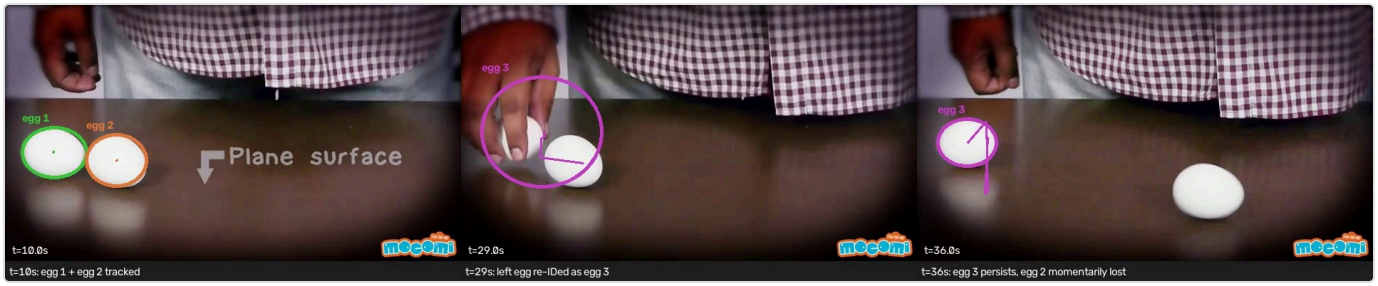


Fig. 7 — Left: both eggs tracked normally. Middle: the hand lifts the left egg for an extended spin demo (~6s); its track survives the occlusion in the tracker's memory but stops being drawn after the grace window. Right: once it reappears, it is just outside the match-and-reappear window and is re-identified as a new ID ("egg 3") rather than recovering "egg 1".

Results

62%	27%	11%	3
frames: both eggs detected	frames: one egg detected	frames: zero eggs detected	distinct track IDs (ideal: 2)

Both eggs are correctly tracked as distinct identities for the large majority of the clip. The one imperfection — 3 total IDs instead of 2 — comes from exactly the ~6.3s occlusion described above: the track survives it, but the egg's position on re-appearance falls just outside the 100px gating distance used for re-matching, so it's assigned a new ID rather than recovering the old one. Identities never swap or merge incorrectly anywhere in the clip, which was the actual objective ("track individual eggs separately").

Full annotated output video: [Drive link pending — see note in submission email]

Setup & reproducing this report

Full setup and run instructions are in the repository's `README.md`. In short: `pip install -r requirements.txt` (torch needs the CUDA 12.1 index explicitly, see README) → `python src/build_manifest.py` → `python src/task1_sam_segment.py` → `python src/task2_dev_experiments.py` → `python src/task3_evaluate.py` → download the video with `yt-dlp` and run `python src/task4_track_eggs.py`.